NIMBUSHOST

Database Normalization Report

Short report covering all 35 tables in the project schema

Purpose. This report explains how the NimbusHost schema organizes data, which tables are deliberately separated, and the normal-form assessment supported by the declared keys and business rules.

Scope. The assessment covers the 35 tables in *database/01_schema.sql*, including transactional tables, operational logs, summary tables, and analytics dimensions/facts.

Executive summary

The schema is largely organized around distinct entities and relationships. Examples include account tiers separated from organizations, operating systems separated from images, ticket headers separated from messages, and invoice headers separated from line items. Primary keys and foreign keys connect these tables without repeating all descriptive details in every row.

Some tables intentionally contain summaries, analytics attributes, JSON payloads, or maintained totals. These are not automatically normalization failures; they are design trade-offs that require clear consistency rules.

> **Important qualification:** a primary key by itself does not prove a table is in 3NF or BCNF. A formal proof requires all relevant functional dependencies and candidate keys. The ratings below are schema-level assessments under the intended business rules, not a mathematical proof of every possible dependency.

1. Normalization in brief

Normal form	What it checks	Project example
1NF	Values are atomic; no repeating groups inside a row.	Separate DNS records are rows in *dns_records*, rather than a list of records in one field.
2NF	In 1NF, and every non-key attribute depends on the whole candidate key—not only part of a composite key.	*org_members*: membership role depends on both *org_id* and *user_id*.
3NF	In 2NF, and no non-key attribute depends transitively on a candidate key.	OS family is kept in *operating_systems*, not repeated with every image/version.
BCNF	For every non-trivial functional dependency X → Y, X is a superkey.	*account_tiers* is BCNF under the declared keys and intended dependencies.

2. Main normalization separations

A. account_tiers → organizations

Tier limits such as maximum instances, storage, and discount belong to a tier, not to each organization using it. *account_tiers* stores those limits once; *organizations.tier_id* references the tier. This reduces duplication and update anomalies. BCNF is supported under the declared keys and dependencies.

B. operating_systems → images

An operating system and a deployable image/version are different concepts. *operating_systems* stores the OS name/family; *images* stores the OS reference, version, and minimum disk requirement. This avoids repeating the OS family for every image and removes the transitive dependency in a combined design. BCNF under the intended dependencies.

C. org_members

This is a junction table for the many-to-many relationship between organizations and users. Its composite primary key is (org_id, user_id). Attributes such as member_role describe that particular membership, so they depend on the full key. This is the key 2NF example.

D. dim_date → dim_month and dim_weekday

dim_date references separate month and weekday tables. Month name/quarter are stored in *dim_month*; weekday name/weekend flag are stored in *dim_weekday*. This removes dependencies between non-key attributes in the date dimension and supports 3NF.

E. invoices → invoice_items

Invoice header information is stored separately from its individual charges. *invoice_items* contains quantity and unit price; the line amount is calculated by a view. This avoids storing the same invoice header details on every line.

F. tickets → ticket_messages

Ticket-level details and conversation messages have different lifecycles and cardinalities. Each message references a ticket, so a ticket can have many messages without repeating the ticket subject/status on every message.

G. instances → instance_type_history

The current instance record is separate from its time-based type history. This lets usage/pricing logic determine which instance type applied at a particular time. The type is intentionally not duplicated in each usage row.

H. domains → dns_records

Domain metadata is stored once in *domains*; each DNS record is a separate row in *dns_records*. This supports multiple records per domain and avoids storing a list of records in one field.

3. Table-by-table assessment — transactional and operational tables

The normal-form column is a practical assessment under the schema's intended functional dependencies. “BCNF under intended FDs” is not a claim that all possible business rules have been formally enumerated.

Table	Key / uniqueness	Assessment	Why / note
account_tiers	tier_id; name is unique	BCNF under declared FDs	Tier limits stored once per tier.
organizations	org_id; name is unique	BCNF under intended FDs	References account_tiers; parent_org_id is a self-reference.
users	user_id; email is unique	BCNF under intended FDs	User/account attributes are stored per user.
org_members	(org_id, user_id)	2NF; also 3NF/BCNF under intended FDs	Composite-key junction table; membership role depends on full key.
api_keys	key_id; key_hash is unique	BCNF under intended FDs	Key metadata is separate from user record; user_id is an FK.
refresh_tokens	token_id; token_hash is unique	BCNF under intended FDs	Refresh-token lifecycle separated from user data.
regions	region_id; code is unique	BCNF under intended FDs	Region metadata is referenced by hosts and IP addresses.
hosts	host_id; hostname is unique	BCNF for row-level attributes	Capacity counters are maintained operational values.
instance_types	type_id; name is unique	BCNF under intended FDs	Hardware specifications are stored once per instance type.
pricing	price_id; (type_id, valid_from) is unique	BCNF under intended temporal rules	Pricing versions are time-scoped rather than overwriting history.
operating_systems	os_id; name is unique	BCNF under intended FDs	Stores OS-level identity and family.
images	image_id; (os_id, version) is unique	BCNF under intended FDs	Stores version-specific image attributes; references OS.
instances	instance_id; idempotency key is unique; live-name rule	BCNF under intended FDs	References organization, host, type, and image; config is JSON.
instance_type_history	(instance_id, valid_from)	BCNF under intended temporal rules	Records the type applicable from each timestamp.
ip_addresses	ip_id; ip_address is unique	BCNF under intended FDs	IP allocation is separate from the instance; nullable instance_id is unique.
volumes	volume_id; live-name rule	BCNF under intended FDs	Volume is a separate resource with its own lifecycle.
snapshots	snapshot_id; (volume_id, name) is unique	BCNF under intended FDs	Snapshots are separate records tied to a volume.
domains	domain_id; name is unique	BCNF under intended FDs	Domain ownership/renewal metadata is stored once.
dns_records	record_id; (domain_id, rtype, name, value) is unique	BCNF under intended FDs	Each DNS record is its own row.
usage_records	(recorded_at, instance_id, metric)	BCNF for stated natural key, with caveat	Partitioned time-series data; no type_id because type is time-dependent. Referential integrity is handled separately due to partitioning constraints.
invoices	invoice_id; (org_id, period_start) is unique	BCNF for row-level FDs; aggregate caveat	Subtotal is a maintained total derived from line items.
invoice_items	item_id	BCNF under intended FDs	Quantity and unit price stored; amount is calculated in a view.
payments	payment_id; txn_ref; idempotency key when present	BCNF under intended FDs	Payment transaction is a separate record tied to invoice.
tickets	ticket_id	BCNF under intended FDs	Ticket metadata separated from its messages.
ticket_messages	message_id	BCNF under intended FDs	Each message is one row and references ticket and sender.

Table	Key / uniqueness	Assessment	Why / note
audit_log	audit_id	BCNF for row-level log records	Old/new record snapshots are JSON; semi-structured by design.
security_log	log_id	BCNF for row-level log records	Event detail is JSON; user_id has no FK by design.
billing_errors	error_id	BCNF for row-level log records	Context is JSON; org_id is nullable and has no FK by design.

4. Table-by-table assessment — summaries and analytics

Table	Key / uniqueness	Assessment	Why / note
daily_usage_summary	(usage_date, org_id)	BCNF for summary row key	Pre-aggregated daily totals; intentionally stores summary metrics.
dim_month	month; month_name is unique	BCNF under calendar rules	Month name and quarter stored once.
dim_weekday	day_of_week; day_name is unique	BCNF under calendar rules	Weekday label and weekend flag stored once.
dim_date	date_key; full_date is unique	3NF/BCNF under calendar rules	References month and weekday dimensions to avoid transitive dependencies.
dim_org	org_key; org_id is unique	Intentional dimensional denormalization	Copies organization/tier/status descriptions for analytical reporting.
dim_type	type_key; type_id is unique	Intentional dimensional denormalization	Copies type name/family/specifications for analytical reporting.
fact_usage_daily	fact_id; (date_key, org_key, type_key, metric) is unique	BCNF under declared keys	Fact table stores aggregated usage by date, organization, type, and metric.

5. Important caveats

• **Derived values:** *v_invoices* calculates invoice tax/total, and *v_invoice_items* calculates line amount. This keeps those calculated values out of base tables where practical.

• **Maintained aggregate:** *invoices.subtotal* can be derived from invoice items, but is stored as a maintained total. This is not automatically a 3NF violation within the invoice row; it does create a cross-table consistency obligation.

• **JSON columns:** *instances.config*, *audit_log.old_data/new_data*, *security_log.detail*, and *billing_errors.context* are semi-structured. If individual JSON attributes become independently managed or heavily queried, consider promoting them to relational columns/tables.

• **Analytics denormalization:** *dim_org* and *dim_type* intentionally duplicate descriptive values; *daily_usage_summary* and *fact_usage_daily* intentionally pre-aggregate data for reporting performance.

• **Usage referential integrity:** *usage_records* is partitioned by time and omits a conventional FK to instances due to MySQL partitioning constraints; integrity is enforced by other database logic, as noted in the schema.

• **Formal proof limitation:** final claims about 3NF/BCNF require a complete list of functional dependencies, candidate keys, and business rules. Unique constraints and comments provide evidence, but do not reveal every possible business dependency.

6. Conclusion

NimbusHost uses a mostly normalized transactional schema: core entities are separated, relationships are represented by foreign keys, composite keys model memberships and time-series facts, and views calculate selected derived values. The clearest normalization examples are *account_tiers*/*organizations*, *operating_systems*/*images*, *org_members*, and the calendar dimensions. Logs, JSON payloads, and pre-aggregated analytics tables are deliberate design choices that should be documented rather than automatically labelled normalization errors.

> **For a viva:** explain the repeated-data problem first, show the original dependency, then show the split tables and their PK/FK relationship. Avoid saying “every table is BCNF” unless all functional dependencies have been checked.